

When to Let a Personal Server Sleep: Keep-Alive Policies for On-Demand Web Apps on a Single User’s Machine

Dhanush Gowdhaman
University of Pennsylvania
dhanushg@seas.upenn.edu

Abstract—A gateway that starts a personal web application when it is first requested and stops it when idle must choose how long to wait: every minute a server stays warm holds memory, every stop risks a cold start the one user feels. Serverless research has studied this keep-alive decision for multi-tenant clouds; we study it for a single user’s laptop. We instrument an on-demand gateway to log every request, cold start and memory sample, build a trace-driven simulator, and validate it against a 6.0-hour live run in which it reproduces the gateway’s 31 cold starts exactly. Across 30 synthetic single-user workloads at three load levels, a fixed 10-minute timeout holds 47% less memory than keeping servers always on (65% at light load) at the cost of about 40 cold starts in six hours. No online policy we evaluate—the hybrid histogram policy from production serverless, per-application learned thresholds, a context-aware variant, and a global user-idle rule—beats that fixed timeout at equal cold starts; all hold 3–14% more memory. An offline oracle holds 42% less (27–56% across loads): the headroom lies in predicting *which* idle gap will be long, which a single user’s gap history does not reveal when visits are memoryless within working and away periods. Freezing idle servers saves no memory, open developer-tool tabs pin their servers indefinitely, and a process manager under-reports a development server’s footprint by 3.8 $\times$. Whether real usage carries the structure these policies need is the question our logged real-usage data will answer.

Index Terms—keep-alive, scale-to-zero, cold start, idle timeout, personal computing, trace-driven simulation

I. INTRODUCTION

On-demand gateways for local web applications—puma-dev [1], Sablier [2], and homegrown ones around process managers such as pm2 [3]—start a server when its hostname is requested and stop it after an idle timeout. The timeout is a trade: warm servers hold memory, which on a laptop is shared with the browser and everything else the user runs; stopped servers cost a cold start on the next visit. In a companion study [4] we measured those cold starts (100–350 ms through the gateway for the applications studied here) and cut them by a median 45%. This paper asks the complementary question: *when should the gateway stop a server at all?*

The cloud version of this question is well studied. Production traces show heavy-tailed, per-function inter-arrival patterns that a histogram of past idle times can exploit [5]; keep-alive has been framed as caching [6] and refined layer by layer [7]. A single user’s machine differs. There is one source of load, whose attention moves between applications and away from the machine altogether; the number of servers is small; each

cold start is felt by the only user; and there is no revenue per invocation, only memory contention with the user’s other work. Mobile operating systems face a similar trade for apps, killing or freezing background processes under memory pressure [8]–[10].

We contribute:

- An **instrumented on-demand gateway** that logs every request (cold or warm, and how long it waited), every start and stop, open WebSockets, and per-service process-tree memory.
- A **trace-driven simulator**, validated against a live run of the real gateway: under the gateway’s own policy it reproduces all 31 cold starts of a 6.0-hour run, per service.
- A **single-user workload model** and an evaluation of fixed timeouts, freezing, the hybrid histogram policy, learned per-application thresholds, context-aware and global user-idle variants, and an offline oracle over 30 workloads at three load levels.
- **Findings** for gateway builders, including two measurement pitfalls that change the answer.

II. SYSTEM, COSTS AND DATA

Gateway. LOCALSERVER routes by hostname to five services of four personal applications (a time tracker’s API and its Vite development server, a project tracker, a job tracker, and a pm2 dashboard), starts a stopped service on the first request, forwards requests once it listens, and stops it after an idle timeout (default 10 minutes). A service with an open WebSocket is never idle.

Usage log. The gateway appends one JSON object per event: each request with its service, whether it arrived cold, how long it waited and its status; each start with its duration and phases; each stop with its reason; each WebSocket open and close; and, every minute, free memory, load, power source and the resident memory of each running service. It records no URLs, paths or addresses. Requests from our traffic generator carry a tag so that simulated and real use can be separated.

Costs. A cold start costs the service’s measured median start time in the live run (105–352 ms). A warm or frozen service holds its process-tree resident memory (66–166 MB). The live run found two pitfalls. First, pm2 reports only a service’s main process: the Vite development server’s real footprint, including the esbuild helper it spawns, is 166 MB, 3.8 $\times$ the 44 MB

pm2 reports. The gateway now samples whole process trees. Second, on the machine we measured (Apple M4 Pro, 24 GB), free memory during the run was a median 0.71 GB and fell to 0.13 GB: memory held by idle servers competes with the user’s own work.

III. WORKLOAD MODEL

We model one user with a two-state modulated Poisson process shared by every application: *working* (visit rates doubled, mean duration 40 min) and *away* (visit rates $\times 0.1$, mean 25 min). Each application has a base visit rate (time tracker 5/h, project tracker 3/h, job tracker 2/h, pm2 dashboard 1.5/h). A visit is a page load (for the development server, the module graph a browser would fetch) followed by $1 + \text{Poisson}(4)$ API calls separated by log-normal think times (median 8 s); with application-specific probability (10–30%) the tab is left open and polls every 30 s for 5–30 minutes. All requests are read-only. The seed-7 trace used for the live run has 64 visits in 6 hours (10.7/h), 17 left-open tabs, and median idle gaps of 8–17 minutes per application—straddling the 10-minute default, which is where the policy decision matters. For the sweep we vary the seed (30 seeds) and scale all rates by $\times 0.5$ (light), $\times 1$ (moderate) and $\times 2$ (heavy).

Within a state, visits in this model are memoryless: nothing in an application’s recent gaps predicts whether its next gap will be long, except through the unobserved state. This is a deliberate null model, and it is the property our findings turn on (§VIII).

IV. SIMULATOR AND VALIDATION

The simulator merges each service’s requests (and open-WebSocket periods) into busy intervals and lets a policy decide every idle gap between them: the next request finds the server warm, frozen, or stopped. Latency is the sum of cold-start (or resume) costs; memory is memory-time, the integral of resident memory of warm and frozen services.

Live run. We replayed the seed-7 trace against the real gateway in real time for 6.0 hours with the machine kept awake: 64 visits, 1097 scheduled requests plus page assets, 0 errors, and 99% of requests sent within 2.7 ms of schedule. The gateway started services 31 times (median 200 ms); no page load needed its “starting” page.

Finding 1. The simulator reproduces the live gateway exactly.

Replaying every request the gateway saw in the window, with each service’s state at the window’s start, the simulator under a fixed 10-minute timeout predicts the same number of cold starts as the live gateway for every service—31 of 31. Two effects had to be modelled to get there, and both matter for policy:

Finding 2. Open developer-tool tabs pin their servers.

Four of the Vite server’s long idle gaps produced no stop. The cause was not a bug: the user had the application open in a browser, and Vite’s hot-reload client keeps a WebSocket open for as long as the tab exists. The gateway rightly treats an open socket as activity, so one forgotten tab held 166 MB for

TABLE I
MEMORY SAVING VERSUS THE FIXED 10-MINUTE TIMEOUT AT ITS OWN NUMBER OF COLD STARTS (% , MEDIAN [IQR] OVER 30 SIX-HOUR WORKLOADS). NEGATIVE: HOLDS MORE MEMORY. [†]THE HISTOGRAM POLICY HAS NO PARAMETER; COMPARED AGAINST THE FIXED FRONTIER AT ITS OWN COLD STARTS.

Policy	light	moderate	heavy
histogram [5] [†]	-5 [-17, 8]	-3 [-9, 0]	-6 [-9, -3]
learned threshold	-11 [-28, -8]	-14 [-20, -9]	-7 [-10, -5]
learned, user-aware	-9 [-22, -4]	-14 [-19, -9]	-6 [-11, -4]
user-idle (global)	-5 [-16, 8]	-7 [-12, -3]	-6 [-10, 0]
oracle (offline)	56 [52, 62]	42 [35, 49]	27 [23, 30]

the last two hours regardless of the timeout. Any keep-alive policy for development servers must decide whether an idle but connected socket counts as use.

With the Vite server’s memory corrected, the live run’s costs are: always on, 2748 MB·h; the 10-minute default, 1374 MB·h (50% less) for 31 cold starts adding 6.9 s of waiting in total; the offline oracle, 939 MB·h (32% less than the default) with 28 cold starts.

V. POLICIES

Fixed timeout T stops a server after T of idleness ($T = \infty$: always on). **Freeze-then-stop** freezes (SIGSTOP) after T_1 and stops after T_2 ; a frozen server resumes in ≈ 3 ms [4] but keeps its memory. **Histogram** [5] learns each service’s idle times in 1-minute bins, unloads after a request unless its 5th-percentile bin is zero, pre-warms at that head, and keeps the server until the 99th percentile, each with a 10% margin; it uses the fixed default until eight idle times are observed. **Learned threshold** chooses, per service and online, the timeout from a grid (0.5–240 min) that would have minimized cold-start latency $+ \lambda \times$ memory-time over the service’s past gaps; λ sets the price of memory. **Learned, user-aware** keeps separate histories depending on whether the user used another application in the 10 minutes before the gap. **User-idle** stops every server once the user has made no request to any application for T_u , and a server once it has been idle for T_a . The **oracle** knows each gap’s length and keeps a server warm through it iff holding its memory costs less than the cold start it avoids, at price λ .

Families with a parameter trace a frontier of (cold starts, memory-time). We compare them at the operating point of the 10-minute default: each family’s memory-time at the default’s number of cold starts, interpolated along its Pareto frontier (achievable by alternating between neighbouring settings), per workload, then median and interquartile range across seeds.

VI. RESULTS

Finding 3. A fixed 10-minute timeout already captures most of the memory saving.

Against keeping every server warm (422 MB on average at moderate load), the default holds 47% less memory-time at moderate load, 65% at light and 30% at heavy load, for a median 40 cold starts in six hours—one wait of a few hundred milliseconds every nine minutes (Fig. 1).

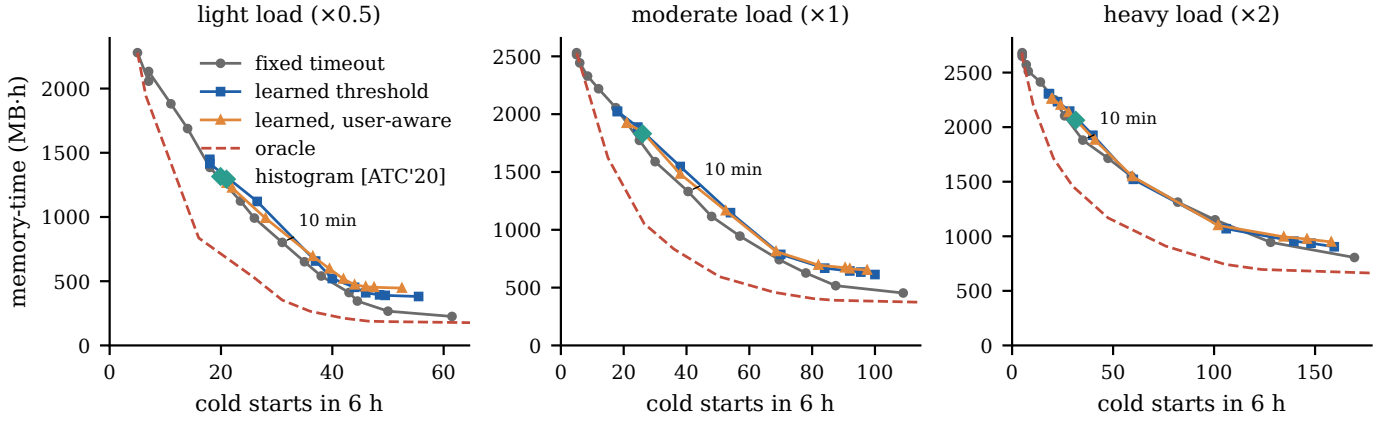


Fig. 1. Memory-time against cold starts over six hours (median across 30 seeds per setting). Lower left is better. Every online policy lies on or above the fixed-timeout frontier; the oracle is far below it.

Finding 4. No online policy beats the fixed timeout at equal cold starts.

At the default’s number of cold starts, the learned threshold holds 14% more memory at moderate load, the user-aware variant 14%, the global user-idle rule 7%, and the histogram policy 3% (Table I); the signs hold at every load level. With eight times the history (48-hour workloads) the learned policies improve but still lose (about -7%), so the shortfall is not only a matter of data. The histogram policy, designed for millions of functions with stable per-function patterns, does not find such patterns in one user’s visits.

Finding 5. The headroom is large, and it is in prediction.

The oracle holds 42% less memory than the default at the same number of cold starts (56% at light, 27% at heavy load). Its advantage comes entirely from knowing, at the start of an idle gap, whether the gap will be long—so that it stops a server at once before a long absence and keeps it through a short one. No policy that sees only past gaps can do that when gaps are memoryless.

Finding 6. Freezing saves no memory.

Freeze-then-stop policies have the same memory-time as the fixed timeout they extend, because a frozen process keeps its resident memory unless the operating system is under enough pressure to compress or swap it [4], [11]; they only turn some cold starts into 3 ms resumes. On a machine that is itself short of memory, the policy that matters is when to stop.

VII. REAL USAGE

Pending real-usage data. Since 29 September 2026 the gateway has logged its user’s real use of these applications. After 2–4 weeks we will (1) characterize real idle gaps per application and across applications (heavy tails, time-of-day and weekday structure, cross-application correlation); (2) test whether gap history or context predicts long gaps (the property Finding 5 shows is needed); and (3) re-run every policy of §V on the real trace with the validated simulator, regenerating Fig. 1 and Table I for real use. This section, the abstract’s last sentence and the conclusion will be updated with those results.

VIII. DISCUSSION

What would make prediction possible. Our workload is a null model: within working and away periods, visits are memoryless, so the only predictable structure is the latent state, and neither a service’s own gaps nor recent activity on other services reveals it well enough to beat a fixed timeout. Real use plausibly has more structure—routines by time of day, applications opened together, open tabs that signal intent—which is exactly what the pending real-usage analysis tests. If it does not, the practical recommendation is simple: pick a fixed timeout by the price of memory on the machine.

Signals a gateway has that a cloud does not. A local gateway can see whether the user’s browser has the application open (connected sockets), whether the machine is short of memory, and whether the user is at the machine at all. Idle-but-connected sockets (Finding 2) are both a signal and a trap: honoring them keeps a forgotten tab’s server warm forever, ignoring them breaks hot-reload. A policy that stops a server whose sockets have been silent for a long time, and lets the client reconnect, is a natural next step.

Threats to validity. The workload is synthetic and its parameters are ours; the live run validates the simulator’s mechanics, not the workload’s realism. Costs come from one machine and five services. Memory-time weights all memory equally, whereas its value depends on what else the user runs; we report the whole frontier so readers can choose their own price.

IX. RELATED WORK

Keep-alive in serverless platforms is characterized on production traces and addressed with histogram-based pre-warming [5], caching policies [6] and layer-wise container caching [7]; complementary work makes cold starts cheaper through lean containers, snapshots and fork-based starts [12]–[19], surveyed in [20]. Deflating idle containers’ memory makes a warm tier cheaper [11]. Mobile systems decide which background apps to kill or freeze under memory pressure [8]–[10]. Local on-demand gateways [1], [2] use fixed timeouts.

We bring the keep-alive question to a single user’s machine, validate a simulator against a live gateway, and show which cloud-derived policies do and do not transfer.

X. CONCLUSION

For personal web applications served on demand, a fixed 10-minute idle timeout halves the memory idle servers hold for a cold start every few minutes, and on synthetic single-user workloads no online policy we tried does better at equal cold starts. An oracle shows that 27–56% more memory could be saved by predicting which idle gaps will be long. Whether a real user’s habits make that prediction possible is the question our gateway’s usage log is now collecting the data to answer.

ARTIFACT

Traffic generator, replayer, simulator, sweep and analysis scripts, the live-run logs and all results are in `keepalive-study/`; `sim/sweep.py` and `sim/analyze_sweep.py` regenerate every figure, table and number.

REFERENCES

- [1] E. Phoenix, “puma-dev: A tool to manage rack apps in development with puma,” <https://github.com/puma/puma-dev>.
- [2] Sablier contributors, “Sablier: Start your containers on demand, shut them down automatically when there is no activity,” <https://github.com/sablierapp/sablier>.
- [3] Keymetrics, “PM2: Production process manager for Node.js,” <https://pm2.keymetrics.io>, version 6.0.13.
- [4] D. Gowdhaman, “Where does the time go? Cold starts of on-demand Node.js web servers on a personal machine,” Companion study (in preparation), 2026.
- [5] M. Shahrad, R. Fonseca, Í. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, “Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2020.
- [6] A. Fuerst and P. Sharma, “FaasCache: Keeping serverless computing alive with greedy-dual caching,” in *Proc. 26th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.
- [7] H. Yu, R. B. Roy *et al.*, “RainbowCake: Mitigating cold-starts in serverless with layer-wise container caching and sharing,” in *Proc. 29th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [8] Android Developers, “Low memory killers,” <https://developer.android.com/topic/performance/vitals/lmk>, accessed Sep. 2026.
- [9] Android Open Source Project, “Cached apps freezer,” <https://source.android.com/docs/core/perf/cached-apps-freezer>, accessed Sep. 2026.
- [10] C. Li, Z. Zhu, C. J. Xue, Y. Liang, R. Ausavarungnirun, L. Shi, and X. Zhou, “Freezing-based memory and process co-design for user experience on resource-limited mobile devices,” *ACM Transactions on Computer Systems*, 2025.
- [11] Y. Sun, D. Vij, F. Li, W. Guo, and Y. Xiong, “Hibernate container: A deflated container mode for fast startup and high-density deployment in serverless computing,” [arXiv:2305.10963](https://arxiv.org/abs/2305.10963), 2023.
- [12] E. Oakes, L. Yang, D. Zhou, K. Houck, T. Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “SOCK: Rapid task provisioning with serverless-optimized containers,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2018.
- [13] D. Du, T. Yu, Y. Xia, B. Zang, G. Yan, C. Qin, Q. Wu, and H. Chen, “Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting,” in *Proc. 25th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.
- [14] A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, “Firecracker: Lightweight virtualization for serverless applications,” in *Proc. 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.
- [15] D. Ustiugov, P. Petrov, M. Kogias, E. Bugnion, and B. Grot, “Benchmarking, analysis, and optimization of serverless function snapshots,” in *Proc. 26th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.
- [16] L. Ao, G. Porter, and G. M. Voelker, “FaaSnap: FaaS made fast using snapshot-based VMs,” in *Proc. 17th European Conference on Computer Systems (EuroSys)*, 2022.
- [17] P. Silva, D. Fireman, and T. E. Pereira, “Prebaking functions to warm the serverless cold start,” in *Proc. 21st International Middleware Conference*, 2020.
- [18] B. Holmes, B. Dinis, L. Honcharuk, J. Fried, and A. Belay, “Taming serverless cold starts through OS co-design,” [arXiv:2509.14292](https://arxiv.org/abs/2509.14292), 2025.
- [19] X. Chai, T. Zhou, K. Hu, J. Tan, T. Bie, A. Shen, D. Shen, Q. Xing, S. Song, T. Yang, L. Gao, F. Yu, Z. He, D. Du, Y. Xia, K. Chen, and Y. Chen, “Fork in the road: Reflections and optimizations for cold start latency in production serverless systems,” in *Proc. 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2025.
- [20] M. Golec, G. K. Walia, M. Kumar, F. Cuadrado, S. S. Gill, and S. Uhlig, “Cold start latency in serverless computing: A systematic review, taxonomy, and future directions,” *ACM Computing Surveys*, 2024.